Optimizing Tool Calling at Scale for LLM-based Agents

Simone Brentan
University of Trento
Trento, Italy

Abstract

This project investigates strategies to optimize tool calling in large-scale LLM-based agents. As the number of available tools increases, naive approaches that load all tool definitions into the model context become infeasible due to token budget constraints, latency, and cost. We explore various tool selection and loading strategies to minimize these issues while maintaining or improving the correctness and reliability of tool usage. Our experiments, conducted across 139 experimental configurations with up to 985 synthetic tools organized in 42 categories, demonstrate that intelligent tool selection mechanisms can significantly reduce context load, from approximately 60,000 tokens down to under 3,000 tokens, while maintaining or even improving tool selection accuracy. We validate our findings using both a custom synthetic dataset and the public xLAM Function Calling 60K benchmark. In particular, implemented tool selection strategies include naive context loading (MCP-style), clustering-based two-step selection, hybrid RAG-based category selection, RAG-based tool selection, and its variant adaptive RAG-based selection. Our results indicate that RAG-based methodologies achieve the best balance between accuracy and efficiency, with adaptive RAG reaching up to 90%+ tool selection accuracy while using less than 5% of the context required by naive approaches.

1 Introduction

Large Language Models (LLMs) have revolutionized the capabilities of AI agents by enabling them to perform complex tasks through tool usage, such as calling APIs, functions, or plugins. However, as the ecosystem of available tools expands to hundreds or thousands, traditional methods that preload all tool definitions into the model's context become impractical. This project aims to address the challenges associated with scaling tool calling in LLM-based agents by exploring and evaluating various tool selection and loading strategies.

Problem Statement. When the tool ecosystem is large and tool documentation is verbose, naive approaches that preload all tool definitions into the model context (MCP-style) become infeasible: they blow the token budget, increase latency and cost, and reduce robustness. For instance, loading 350 tools with medium-verbosity descriptions consumes approximately 60,000 tokens, nearly exhausting the 64K context window of models like Llama 3.3 70B, leaving minimal room for conversation history, system instructions, or user queries. Beyond this threshold, model performance degrades significantly as attention mechanisms struggle with the overwhelming context. We study selection and loading strategies that minimize token/compute cost and latency while preserving (or improving) correctness and reliability of tool usage. The core research question is: **Can we achieve comparable or better tool selection accuracy while loading only a small fraction of available tools into context?**

Motivation. Real-world deployments (enterprise skill stores, platform plugins, multi-tenant APIs) contain many overlapping implementations (multiple mail providers, multiple browsing adapters, etc.). As organizations adopt LLM-based agents for automation, the tool ecosystem naturally grows: a typical enterprise might have dozens of internal APIs, hundreds of SaaS integrations, and thousands of individual functions available for the agent to invoke.

This creates several challenges:

- **Token Economics**: With API pricing often based on token consumption, inefficient tool loading directly impacts operational costs. Loading all tools for every request can multiply inference costs by an order of magnitude.
- **Latency**: Larger contexts increase inference time, degrading user experience in interactive applications.
- **Accuracy Degradation**: Empirically, we observe that LLMs struggle to select the correct tool when presented with similar options, leading to increased error rates.
- **Context Competition**: Tokens consumed by tool definitions reduce the budget available for conversation history, few-shot examples, and detailed instructions.

Efficient and accurate tool selection reduces operational cost, improves uptime and user experience, and is fundamental for scaling agent systems in production.

Methodologies Explored. We conducted a series of experiments to evaluate different tool selection strategies under varying conditions of tool count and documentation verbosity.

The study includes the following configurations:

- **Naive Context Loading**: All tool definitions are loaded into the model context, serving as a baseline for comparison.
- **Clustering-Based Two-Step Selection**: Tools are clustered, and a two-step selection process is employed, first selecting a cluster and then a specific tool within that cluster.
- **Hybrid RAG-Based Category Selection**: A hybrid approach using Retrieval-Augmented Generation (RAG) for category selection to inject into context.
- **RAG-Based Tool Selection**: Directly selecting top-k tools using RAG to minimize context load.
- **Adaptive RAG-Based Selection**: An adaptive variant of RAG-based selection that dynamically adjusts the number of tools selected based on the query.

2 Related Work

The challenge of scaling tool usage in LLM-based agents has garnered significant attention in recent research, though most work focuses on tool learning rather than efficient tool selection at scale.

2.1 Function Calling in LLMs

OpenAI's function calling capability, introduced in GPT-3.5 and GPT-4, established the modern paradigm of structured tool invocation where models output JSON-formatted function calls. However, OpenAI's approach assumes a relatively small set of tools passed directly in the context. Similarly, Anthropic's Claude and Google's Gemini support tool use but face the same context limitations. The Model Context Protocol (MCP) standardizes tool definition formats but does not address selection at scale.

2.2 Tool Learning and Selection

ToolFormer (Schick et al., 2023) [6] pioneered self-supervised tool learning, training models to decide when and how to use tools. However, it focuses on a small fixed toolset. Gorilla (Patil et al., 2023) [3] introduced a fine-tuned LLaMA-based model specifically optimized for writing API calls, surpassing GPT-4's performance on this task. Crucially, Gorilla demonstrated that integrating a document retriever enables adaptation to test-time document changes, allowing flexible updates to API documentation and substantially mitigating hallucination issues common in direct LLM prompting. Their accompanying APIBench dataset, covering HuggingFace, TorchHub, and TensorHub APIs, established a comprehensive evaluation framework for API-based tool use.

ToolBench (Qin et al., 2023) [4] introduced a large-scale benchmark with 16,000+ APIs and proposed a tree-based reasoning strategy. While comprehensive, their focus is on complex multi-step tool chains rather than single-tool selection efficiency.

Pan et al. (2025) [2] recently introduced Online-Optimized RAG, a deployment-time framework that addresses embedding misalignment issues in RAG-based tool selection. Their approach applies lightweight online gradient updates using minimal feedback (e.g., task success) to continuously adapt retrieval embeddings from live interactions. Notably, this method requires no changes to the underlying LLM and supports both single- and multi-hop tool use with dynamic tool inventories. Their work provides a complementary direction to our static evaluation, demonstrating that RAG-based tool selection systems can be further improved through online adaptation.

2.3 Retrieval-Augmented Generation

RAG systems (Lewis et al., 2020) [1] have become fundamental for grounding LLM responses in external knowledge. Dense passage retrieval using sentence transformers (Reimers & Gurevych, 2019) [5] enables semantic similarity search across large document collections. We adapt these techniques for tool description retrieval, using embeddings to identify semantically relevant tools before LLM invocation.

Our Contribution. While prior work focuses on improving individual tool usage capabilities (Gorilla) or online adaptation of existing systems (Pan et al., 2025), we specifically address the efficiency-accuracy trade-off in tool selection at scale. Our systematic comparison of five methodologies across varying scale provides practical guidance for deploying LLM agents in production environments with large tool ecosystems. This offline evaluation of selection strategies complements online adaptation approaches and informs initial system design choices.

3 Experimental Design Overview

Before detailing each methodology, we outline the experimental framework used throughout this study.

Experimental Assumptions. A key simplifying assumption guides our experiments: **tools do not need to be functionally implemented; they only need to be correctly selected**. This allows us to define tools as YAML configuration files without backend implementations, focusing evaluation purely on selection accuracy. This assumption is valid for studying tool routing strategies, though real deployments would require full implementations.

3.1 Language Models Used

We conducted experiments with two LLM configurations:

Llama 3.3 70B (Cloud via Cerebras API):

- Context window: 64,000 tokens
- Primary model for methodology development and analysis
- Free API tier with rate limiting, restricting us to 10 samples × 3 seeds per configuration
- Used for majority of results presented

Llama 3.2 3B (Local via Ollama):

- Context window: 4,096 tokens
- Used for complete validation runs testing all tools in configuration files
- Smaller model allows faster iteration and full dataset evaluation
- Validates that trends observed with larger models hold for smaller ones

3.2 Metrics Justification

We evaluate each methodology using carefully selected metrics that capture different aspects of tool selection performance:

Tool Selection Accuracy *(Primary Metric).* The percentage of queries where the correct tool was selected. This is the fundamental measure of system utility. If the wrong tool is selected, the agent cannot complete the user's request. We consider an exact match between the predicted tool name and the expected tool name.

Category Selection Accuracy *(Hierarchical Methods).* For clustering-based and hybrid approaches, we separately measure how often the correct category is selected in the first step. This diagnostic metric helps identify whether errors originate in category selection or tool selection within categories.

Context Size *(Input Tokens).* The number of tokens consumed by tool definitions in the model's input. This directly correlates with:

- API costs (most providers charge per token)
- Inference latency (attention scales quadratically with context length)
- Remaining context budget for conversation history and instructions

We measure this using the LLM provider's token counting mechanism to ensure accuracy.

Latency. End-to-end wall-clock time from receiving a query to returning a tool selection (in milliseconds). This includes:

- Embedding computation (for RAG methods)
- Similarity search
- LLM inference
- Any multi-step reasoning (for hierarchical methods)

Retrieval Recall *(RAG Methods).* For RAG-based approaches, whether the correct tool was present in the retrieved candidate set. High retrieval recall with lower final accuracy indicates the LLM is struggling with selection from candidates; low retrieval recall indicates the embedding-based retrieval is the bottleneck.

4 Methodologies

In this section, we detail the five tool selection strategies implemented and evaluated in our experiments. For each methodology, we describe the approach, implementation details, and experimental results. All primary results shown are based on experiments conducted with the Llama 3.3 70B model via cloud API.

4.1 Tool Definition & Dataset

4.1.1 Synthetic Tool Dataset. We constructed a synthetic dataset of **985 tools** organized into **42 categories** covering common enterprise and developer use cases. The dataset was designed to simulate realistic tool ecosystems with intentional challenges:

Each tool is defined in a *YAML* file with the following schema:

```yaml
- name: read_file
  descriptions:
    minimal: "Read a file"
    short: "Read the contents of a file from disk"
    medium: "Read the contents of a file from ..."
    long: "Read the contents of a file from ..."
    verbose: "Read the contents of a file from ..."
  parameters:
    - name: file_path
      type: string
      description: "The path to the file to read"
      required: true
    - name: encoding
      type: string
      description: "The file encoding (default: utf-8)"
      required: false
  tags: ["file", "read", "io", "filesystem"]
  test_prompts:
    single:
      - "Read the contents of config.json"
      - "Show me what's in the README.md file"
    single_clear:
      - "I need to read and display the text contents ..."
```

Each tool includes five description verbosity levels (minimal, short, medium, long, verbose) enabling experiments on how description detail affects selection accuracy. Verbose descriptions include usage examples, warnings, and edge cases.

Test Prompts: Tools include two types of test prompts. The *single* type consists of concise, natural language prompts (e.g., "Read config.json"), while the *single_clear* type uses explicit prompts that clearly state the intent (e.g., "I need to read and display text contents of a file from the filesystem. Read config.json").

Ambiguity by Design: The dataset includes intentionally similar tools to test disambiguation capabilities. Examples include *manage_connection_pool* vs. *manage_database_connection_pool*, *send_email* vs. *send_notification* vs. *send_message*, and multiple tools with overlapping functionality across categories.

Category Distribution: Tools are distributed across 42 categories including file operations, database operations, ai/ml operations, email operations, authentication operations, and others. Each category contains 15-40 tools with related functionality.

4.1.2 xLAM Function Calling 60K Dataset. To validate our findings on realistic data, we use the **xLAM Function Calling 60K** dataset, a public benchmark containing verifiable high-quality API descriptions and natural language queries. This dataset provides diverse API descriptions from actual services, naturally phrased user queries, and ground truth tool selections for evaluation.

Results on this dataset confirm that trends observed with our synthetic dataset generalize to real-world scenarios.

4.1.3 Prompt Construction. When presenting tools to the LLM, we format each tool as a structured function definition following OpenAI's function calling convention. The system instruction guides the model to select the most appropriate tool:

You are selecting tools to help complete user requests. Given the user's query, choose the most appropriate tool from the available options. Only call one tool that best matches the user's intent.

For methodologies that include pseudo-tools (e.g. *__backtrack__* for clustering-based technique), these are presented alongside regular tools with special descriptions explaining their purpose.

4.2 Naive Context Loading

The naive context loading approach involves preloading all tool definitions into the model's context. This is the simplest method but is also the style commonly used by the MCP protocol. This method serves as a baseline for comparison against more sophisticated selection strategies.

Different experiments are conducted with an increasing number of tools: 10, 25, 50, 100, 200, 300, 350. It was impossible to test with 400 tools due to context window limitations of the LLM used. In fact, with 400 tools the context size exceeded the model's maximum token limit and all API requests failed.

4.2.1 Results. Figure 1 line chart shows tool selection accuracy (y-axis, 0-100%) as the number of tools increases (x-axis, 10-350). Accuracy remains high (90-95%) for configurations up to 300 tools, then drops sharply at 350 tools where context saturation begins to impact model performance.

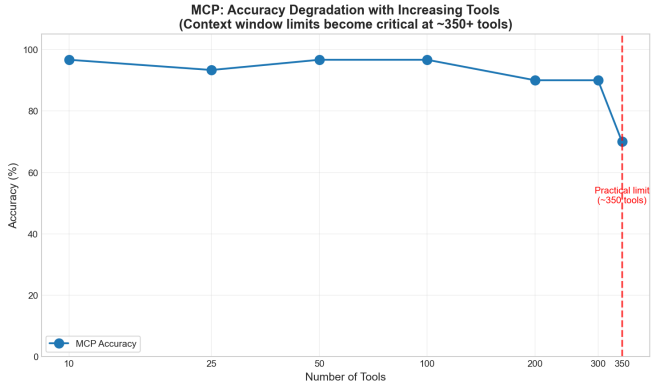

Figure 1: MCP Tool Selection Accuracy vs. Number of Tools.

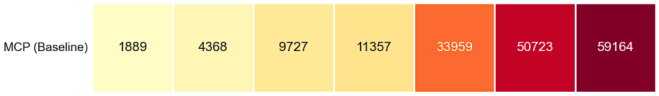

Figure 2: MCP Input Token Usage Heatmap

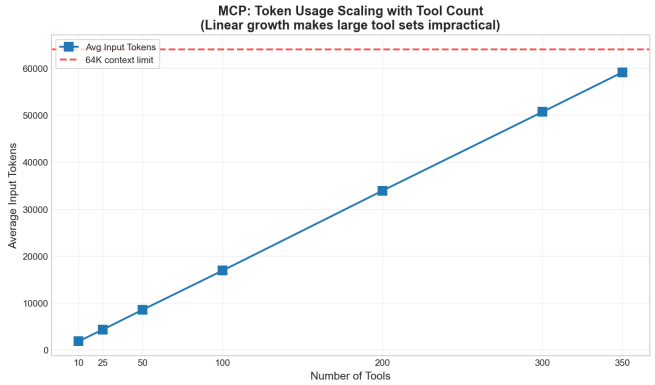

Figure 3: MCP Context Size (Input Tokens) vs. Number of Tools.

Figure 2 and 3 charts demonstrates the linear relationship between tool count and token consumption. Each tool consumes approximately 150-180 tokens (with medium verbosity), leading to 60,000 tokens at 350 tools, near the 64K context limit of Llama 3.3 70B. In the input token usage heatmap, the shaded region indicates the "danger zone" where context pressure begins affecting accuracy.

These figures illustrate the fundamental scalability problem: while MCP achieves high accuracy when context permits, token consumption grows linearly and eventually exhausts the available context window. Beyond 350 tools, API requests failed entirely due to context overflow.

4.3 Clustering-Based Two-Step Selection

In order to address the scalability issues observed with naive context loading, we implemented a clustering-based two-step selection strategy. This approach involves first clustering tools into categories based on their functionality, and then performing a two-step selection process:

- **Category Selection**: The model first selects the most relevant category of tools based on the user query.
- **Tool Selection**: Within the selected category, the model then selects the specific tool to invoke.

This method aims to reduce the context size by only loading tools from the selected category into the model's context. However, this approach introduces complexity in accurately selecting the correct category, which can impact overall tool selection accuracy. In addition, compared to naive context loading, this method requires two separate calls to the LLM: one for category selection and another for tool selection within that category, potentially increasing latency.

For this methodology, tests included tools ranging from 10 to 200 tools, similarly to the naive context loading, and then two additional layers of tests with respectively 500 and 985 tools.

Note: Although this methodology involves clustering tools, in this project we did not implement an automatic clustering algorithm but we manually defined the categories and assigned tools to them based on their functionality. This is a simplification to focus on evaluating the two-step selection process itself rather than the clustering quality.

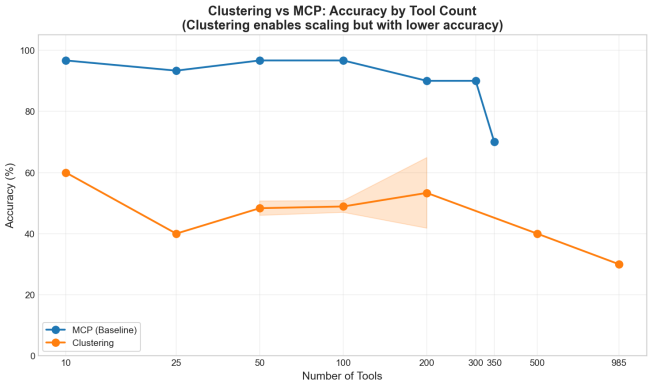

Figure 4: Clustering Tool Selection Accuracy vs. Number of Tools.

4.3.1 Results. This chart shows that clustering-based selection maintains relatively stable but low accuracy (40-50%) across all tool counts from 10 to 985. Unlike MCP, accuracy does not degrade drastically with scale because context size remains bounded by category size. However, the baseline accuracy is significantly lower due to category selection errors.

Figure 5 shows that context consumption remains bounded regardless of total tool count, typically staying under 10,000 tokens. This is because only tools from the selected category are loaded, a significant reduction from MCP's linear growth. The graphs reveal the core trade-off: clustering dramatically reduces context usage but introduces a bottleneck at the category selection step.

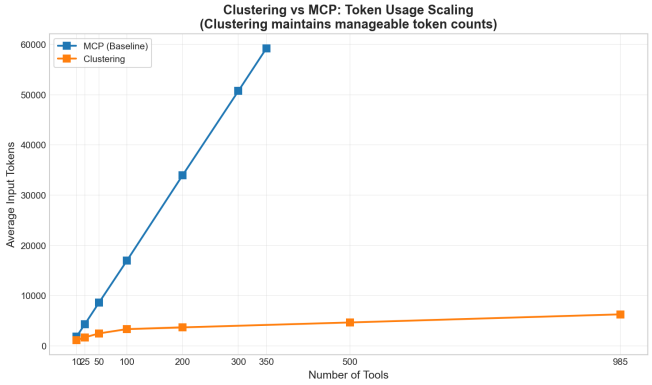

Figure 5: Clustering Context Size vs. Number of Tools.

The accuracy degradation is primarily due to the model's difficulty in correctly identifying the relevant category from the user query. The accuracy remains relatively stable and low about 40-50% across different tool counts due to how the tests have been designed. Even when creating a test with only 10 tools, they are chosen from different categories, therefore the model still needs to correctly identify the category first, which is challenging.

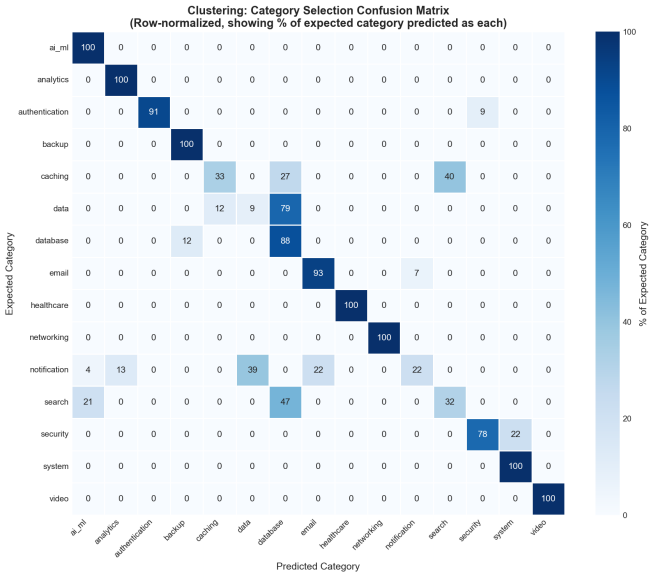

Figure 6: Category Selection Confusion Matrix (Clustering).

Figure 6 heatmap shows the relationship between expected categories (y-axis) and predicted categories (x-axis) for the top 15 most frequently tested categories. Darker cells on the diagonal indicate correct predictions; off-diagonal darkness reveals systematic confusion patterns. Notable confused categories include the search category and notification category, frequently mistaken with other categories due to being broad in scope.

Note: Only the most-used 15 categories are displayed for clarity; the full matrix covers all 42 categories.

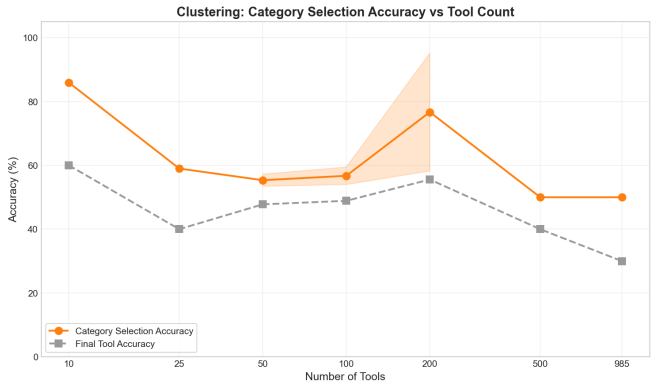

Figure 7: Category Selection Accuracy vs. Number of Tools.

Figure 7 line chart shows category selection accuracy (y-axis, 0-100%) as the number of tools increases (x-axis, 10-985). The other dashed line shows the overall tool selection accuracy for reference (same as Figure 4). It can be seen that, while the category selection accuracy is far from ideal, there is still a significant gap between it and the overall tool selection accuracy, indicating that even when the correct category is selected, the model often fails to select the correct tool within that category. This indicates that a more studied prompt engineering for both category and tool selection steps is required to improve the overall accuracy of this methodology.

Additional design choices. The category selection prompt defines the list of categories available for selection as a list of tools that the model can call. For example, if we have 3 categories: "AI ML Operations", "Analytics Operations" and "Database Operations", the prompt will define these categories as 3 tools with their respective descriptions:

- *select_category_ai_ml_operations*: "AI and machine learning operations including text generation, classification, embeddings, model inference, and natural language processing"
- *select_category_analytics_operations*: "Data analytics, statistics, reporting, dashboards, and business intelligence operations"
- *select_category_database_operations*: "Database queries, CRUD operations, schema management, migrations, and data retrieval from SQL/NoSQL databases"

To further improve the category selection accuracy, a **backtrack** mechanism has been implemented. To the list of categories defined as tools in the prompt, an additional tool called *__backtrack__* is added that enables the model to return back to the previous step if it realizes that the selected category was incorrect. A maximum number of 10 max steps (5 backtracks) is allowed to avoid infinite loops. The list of previously selected categories is also provided in the prompt to avoid repeating the same mistakes.

As a secondary test, also the list of tool names have been added in each category tool description, but this did not change the accuracy significantly.

Even though it was provided, backtracking was used very rarely, highlighting the model's difficulty in selecting a category by calling the correct tool. More tests are required to effectively test the

clustering methodology, for example by improving the prompt and tool definitions or descriptions. However, the results of this initial implementation already show that the category selection step is a significant bottleneck for this approach, and that improving it is crucial for achieving better accuracy while maintaining context efficiency.

4.4 Hybrid RAG-Based Category Selection

After seeing the limitations of the clustering-based two-step selection, particularly in terms of category selection accuracy, we explored a hybrid approach that leverages Retrieval-Augmented Generation (RAG) for category selection.

In this hybrid RAG-based category selection method, the model uses RAG to select the most relevant categories based on the user query. The selected categories are then injected into the model context as tools, similar to the clustering-based approach. This allows the model to focus on a smaller subset of tools while benefiting from the retrieval capabilities of RAG.

This solution is a hybrid between full clustering-based and full RAG-based tool selection, as this methodology uses RAG to select the categories, and then uses the selected categories to load tools into context (like clustering-based two-step selection).

This methodology introduces an additional layer of complexity, as it relies on the effectiveness of RAG in accurately retrieving relevant categories. The RAG component for category selection uses a vector store built from the tool categories' descriptions. When a user query is received, the RAG system retrieves the top-k most relevant categories based on similarity to the query.

For this methodology, tests included tools ranging from 10 to 985 tools, similarly to the clustering-based two-step selection.

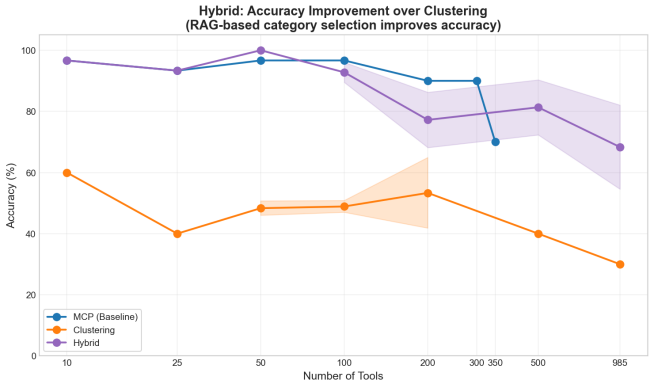

Figure 8: Hybrid RAG Tool Selection Accuracy vs. Number of Tools.

4.4.1 Results. This chart shows substantial improvement over pure clustering, with accuracy ranging from 70 to 100% depending on configuration. The hybrid approach recovers much of the accuracy lost in clustering by using semantic similarity for category selection instead of forcing the LLM to choose from category descriptions alone. RAG embeddings are able to capture categories descriptions semantically, therefore the model can better understand the relationship between user queries and relevant categories. On the other hand, accuracy remains generally below MCP for all configurations where MCP does not hit context limits. This indicates that category selection still remains a bottleneck for not finding all matching tools to pass to the model.

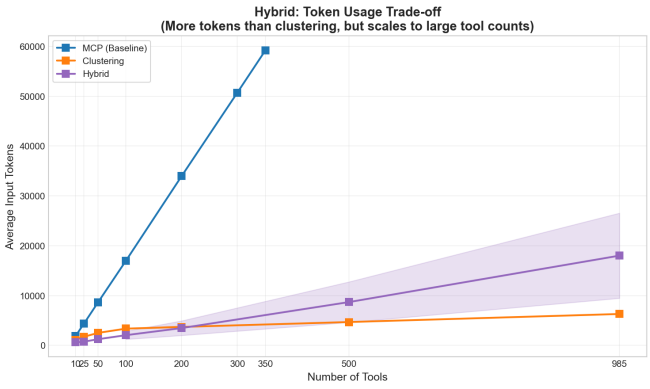

Figure 9: Hybrid RAG Context Size vs. Number of Tools.

Token consumption remains bounded but is higher than pure clustering since multiple categories may be retrieved (top-k categories × tools per category). With k=3-5 categories, typical context size ranges from 10,000-20,000 tokens.

The results demonstrate that hybrid RAG substantially improves accuracy over clustering while maintaining context efficiency comparable to MCP.

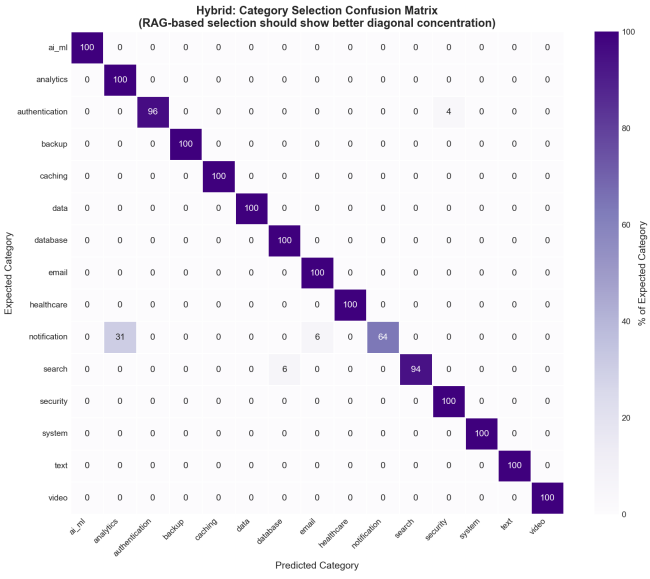

Figure 10: Category Selection Confusion Matrix (Hybrid RAG).

Compared to the clustering confusion matrix (Figure 6), this heatmap shows stronger diagonal concentration, indicating improved category selection accuracy through RAG-based retrieval. The embedding-based approach better captures semantic similarity between queries and category descriptions.

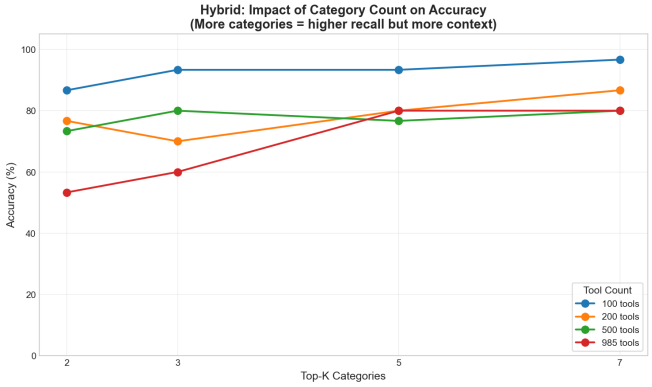

Figure 11: Impact of Retrieved Category Count (k) on Accuracy.

Figure 11 shows how varying the number of retrieved categories affects tool selection accuracy. Increasing k from 3 to 5 yields significant accuracy gains, particularly for the 985-tool configuration. Beyond k=5, diminishing returns are observed as context load increases without proportional accuracy improvements. Since each category contains 30-40 tools on average, k=5 means loading 150-200 tools.

Implementation Details. The RAG component uses **Sentence Transformers** with the *all-MiniLM-L6-v2* embedding model, a compact (22M parameters) but effective model optimized for semantic similarity tasks. This choice balances embedding quality with computational efficiency, as embeddings must be computed for each query at inference time.

For the hybrid approach, categories can be embedded using two strategies:

- **Description-based**: Embed the category description text directly
- **Mean pooling**: Compute the average embedding of all tools within each category

We use description-based embedding by default, as category descriptions are crafted to be semantically representative. The improved category accuracy compared to clustering partially stems from RAG's ability to leverage the rich semantic information in these descriptions, whereas the LLM in clustering only sees category names and brief descriptions.

All experiments use the same embedding model for consistency. Evaluating alternative embedding models (e.g., larger models like *all-mpnet-base-v2* or domain-specific models) represents a potential avenue for future optimization.

4.5 RAG-Based Tool Selection

Building upon the insights gained from the hybrid RAG-based category selection, we implemented a full RAG-based tool selection strategy. With this approach, we aim to skip the category selection step and directly select the most relevant tools using RAG, thereby minimizing context load while maximizing tool selection accuracy. Context load is minimized by only injecting the top-k tools retrieved by RAG into the model context, instead of loading all tools for a subset of categories. Additionally, this methodology aims to retrieve tools that may belong to different categories but are all relevant to the user query.

For this methodology, tests included tools ranging from 10 to 985 tools, similarly to the previous Clustering and Hybrid methodologies. The RAG component for tool selection uses a vector store built from the tool definitions, including their names and descriptions. When a user query is received, the RAG system retrieves a fixed number of the most relevant tools based on similarity to the query. The number of tools retrieved (k) depends on the specific test configuration and is chosen to balance context size and accuracy.

Note: The RAG system uses the same embedding model and parameters as the hybrid RAG-based category selection for consistency across experiments.

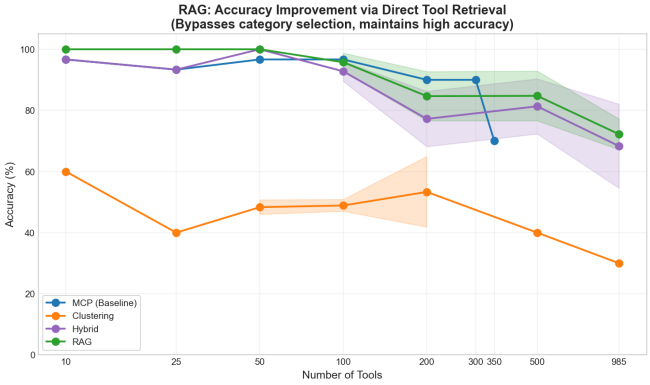

Figure 12: RAG Tool Selection Accuracy vs. Number of Tools.

4.5.1 Results. Figure 12 shows that RAG-based selection maintains high accuracy (80-100%) across all tool counts from 10 to 985. Unlike clustering, there is no category selection bottleneck. Unlike MCP, there is no context saturation. The direct retrieval of semantically relevant tools proves highly effective.

Token consumption shown in Figure 13 remains nearly constant regardless of total tool count, a key advantage of the RAG approach. With k=10, context size stays around 1,500-2,000 tokens; with k=15, approximately 2,500-3,000 tokens. This represents a 20-30x reduction compared to MCP at 350 tools.

The results demonstrate that RAG achieves the best of both worlds: accuracy comparable to or exceeding MCP with context efficiency far superior to all other approaches.

Two additional analysis were conducted in this methodology:

- Impact of changing the number of tools retrieved by RAG (k) on overall tool selection accuracy.
- Retrieval recall analysis to understand how often the correct tool is present in the retrieved candidate set.

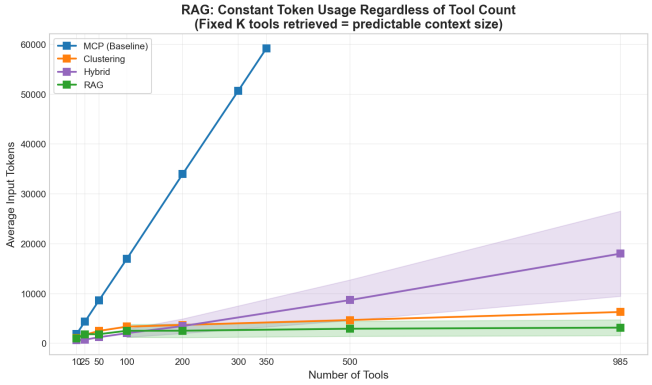

Figure 13: RAG Context Size vs. Number of Tools.

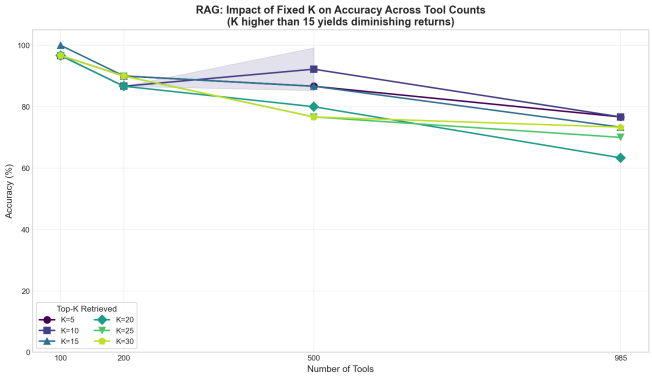

Figure 14: Impact of k (Retrieved Tools Count) on RAG Accuracy.

In the first analysis, shown by Figure 14, k varies from 5 to 30 across different total tool counts. Optimal accuracy occurs at k=10-15; beyond k=15, accuracy slightly decreases as the model becomes confused by multiple similar tools in context. This suggests that "more is not always better", targeted retrieval outperforms broader retrieval.

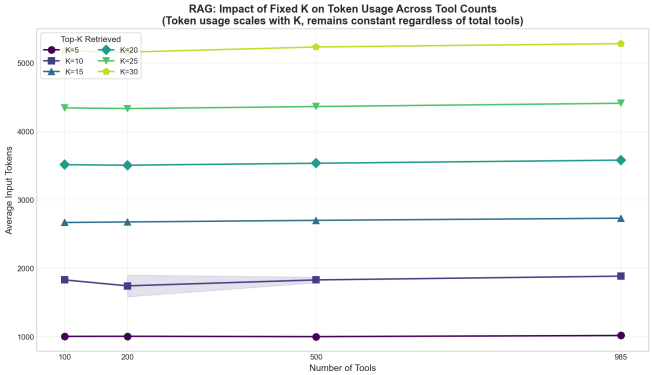

Figure 15: Impact of k on RAG Context Size.

Figure 15 instead shows that context size scales linearly with k but remains independent of total tool count. At k=30, approximately 5,500 tokens are consumed; at k=15, around 2,700 tokens; at k=10, approximately 1,800 tokens. Even at the highest k tested, context usage remains well below MCP's requirements.

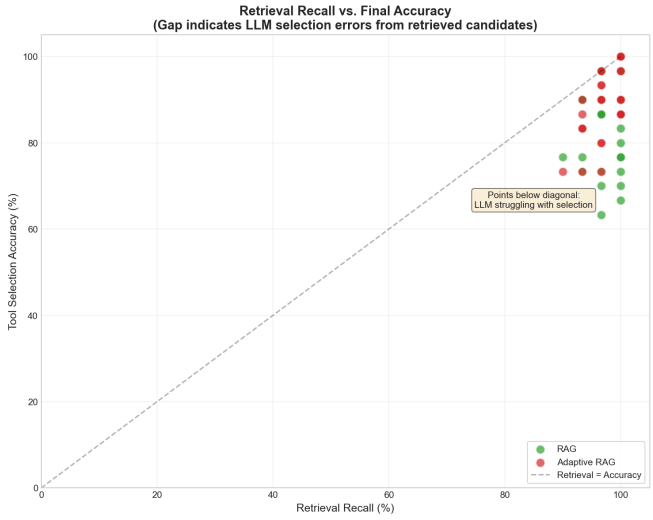

Figure 16: Retrieval Recall vs. Final Accuracy Scatter Plot.

The scatter plot of Figure 16 compares retrieval recall to final tool selection accuracy for each configuration. Points on the diagonal line indicate that retrieval recall equals accuracy (perfect LLM selection from candidates). Points below the diagonal indicate the LLM is struggling to select the correct tool even when it is present in the candidate set. Both Adaptive RAG and fixed-k RAG configurations have a high retrieval recall (above 90%), indicating that the retrieval is effective. The gap between recall and accuracy highlights opportunities for improving LLM selection capabilities.

4.6 Adaptive RAG-Based Selection

Building upon the RAG-based tool selection, we implemented an adaptive variant that dynamically adjusts the number of tools (k) retrieved based on the similarity score distribution for each query.

Rather than using a fixed k value, adaptive RAG analyzes how similarity scores are distributed and selects k accordingly, retrieving fewer tools when the query clearly matches a small set of tools, and more tools when multiple candidates have similar relevance.

4.6.1 Adaptive K Selection Algorithm. The adaptive k selection uses three complementary strategies combined with bounds:

Strategy 1: Elbow Detection. Sorts tools by descending similarity score and computes the gradient (difference) between consecutive scores. The algorithm identifies the first "significant drop" where the gradient exceeds a threshold (default: 0.1). This point represents a natural boundary between highly relevant and less relevant tools.

```
sorted_similarities = sort(
    similarities, descending=True
)
gradients = diff(sorted_similarities)
for i, gradient in enumerate(gradients):
    if i>=min_k-1 and gradient>=drop_threshold:
        elbow_k = i + 1
        break
```

Strategy 2: Threshold-Based. Counts tools with similarity scores above a minimum threshold (default: 0.3). This ensures a baseline quality level for all retrieved tools.

Strategy 3: Combined Bounds. Takes the more restrictive (smaller) of elbow_k and threshold_k, then applies bounds:

- **min_k** (default: 3): Ensures at least some tools are always retrieved
- **max_k** (default: 20): Prevents excessive context consumption

The final k is computed as: *final_k = max(min_k, min(combined_k, max_k))*

Note: This approach is a variant of standard RAG rather than a fundamentally new methodology. It was developed to explore whether dynamic k selection could improve upon fixed k values.

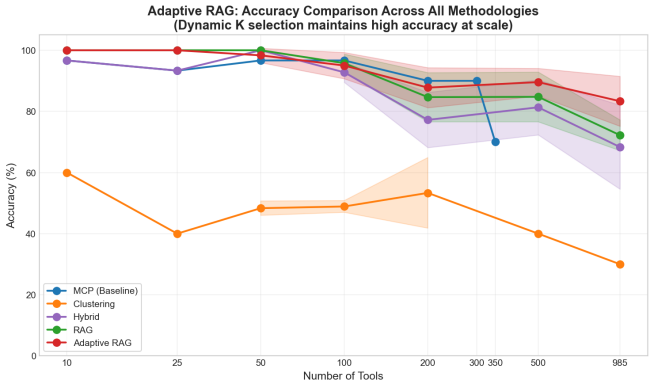

Figure 17: Adaptive RAG Tool Selection Accuracy vs. Number of Tools.

4.6.2 Results. Adaptive RAG achieves accuracy comparable to or slightly higher than fixed-k RAG, particularly in higher tool count configurations (500-985 tools). The dynamic k selection helps in reducing both under-retrieval (missing the correct tool) and over-retrieval (confusing the LLM with too many options).

Token consumption varies more than fixed-k RAG (as shown in Figure 19) due to the dynamic nature of k selection. The adaptive algorithm selects fewer tools when queries are unambiguous and more tools when ambiguity exists, leading to context sizes ranging from 1,000 to 5,000 tokens depending on the query.

Figure 20 shows the distribution of k values selected by the adaptive algorithm across all test queries. The majority of queries result in minimum or maximum k values (3 and 20), indicating that

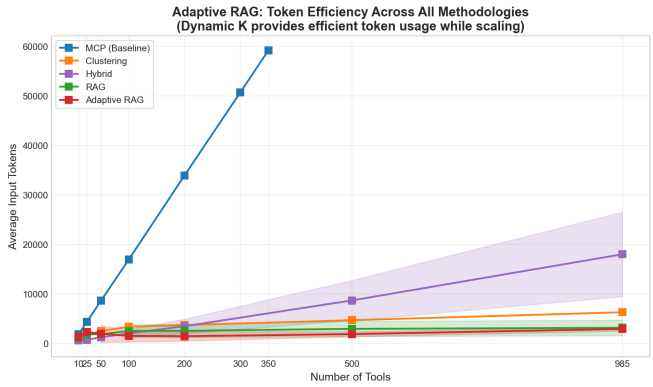

Figure 18: Adaptive RAG Context Size vs. Number of Tools.

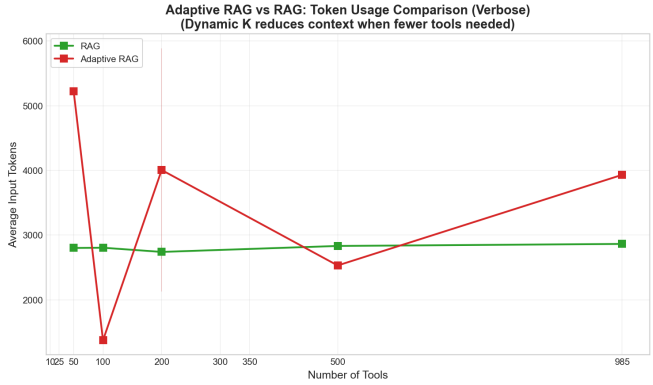

Figure 19: Adaptive RAG vs. Fixed RAG Token Usage Comparison.

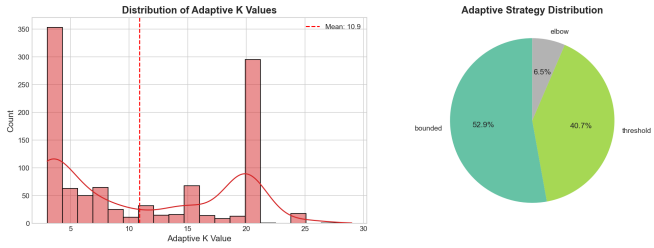

Figure 20: Adaptive RAG Retrieved k Value Distribution.

many queries are either very clear or very ambiguous. However, a significant portion of queries yield intermediate k values, demonstrating the algorithm's ability to tailor retrieval depth based on query characteristics. More fine-tuning of the adaptive parameters (drop threshold, similarity threshold, min/max k) could further optimize performance. This can be explored and validated in future work.

Prompt Clarity Analysis. Preliminary experiments compare "single" (concise) prompts versus "single_clear" (explicit) prompts. The difference of explicite and concise prompts is simply that explicit

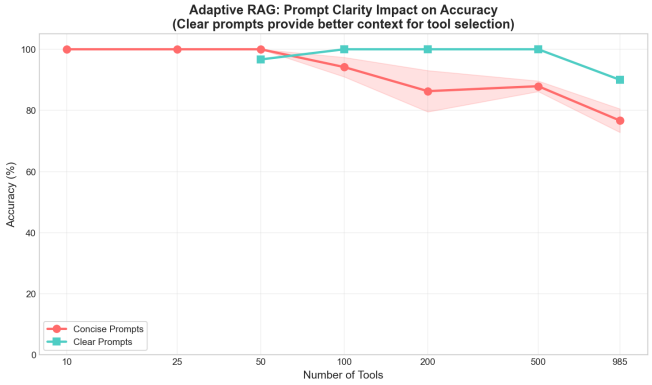

Figure 21: Impact of Prompt Clarity on Adaptive RAG Accuracy.

prompts try to better point to the correct category and tool to select. This is a simple test and major improvements can be made to the prompt engineering to further improve accuracy.

As shown in Figure 21, clear prompts show improved accuracy, particularly at higher tool counts where disambiguation becomes more challenging. However, this analysis requires additional testing to draw definitive conclusions.

5 Experiments & Analysis

This section consolidates findings from all experiments and provides cross-methodology comparisons. We conducted a total of **139 experimental configurations** across the five methodologies, varying parameters such as tool count, verbosity level, retrieval k values, and prompt clarity.

Table 1: Methodology Evaluation and Parameter Variance

Methodology	Tool Range	Key Parameters
MCP	10 - 350	Verbosity (short/verbose)
Clustering	50 - 985	Backtrack enabled/disabled
Hybrid RAG	100 - 985	k_categories: 2-7
RAG	100 - 985	top_k: 5-30, similarity: 0.0-0.2
Adaptive RAG	200 - 985	min_k: 3-5, max_k: 15-25

5.0.1 Experimental Scope. Each configuration was tested with 10 samples × 3 random seeds (30 total runs) where API limits permitted. Due to free-tier rate limiting on cloud LLM APIs, some high-volume configurations were run with fewer iterations.

5.0.2 Overall Methodology Comparison. Heatmap of Figure 22 provides a comprehensive view of tool selection accuracy (color-coded, darker = higher accuracy) across all methodologies (rows) and tool counts (columns). Key observations:

- **MCP**: High accuracy (>90%) up to 300 tools, then unavailable beyond 350 due to context limits
- **Clustering**: Consistently low accuracy (40-50%) across all tool counts

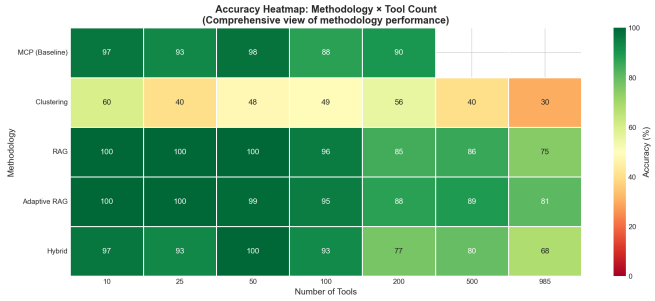

Figure 22: Accuracy Heatmap - Methodology vs. Number of Tools.

- **Hybrid RAG**: High accuracy but scales poorly beyond 200/500 tools
- **RAG**: High accuracy (80-90%+) that scales to 985 tools
- **Adaptive RAG**: Highest accuracy in most configurations, particularly at scale

The heatmap clearly illustrates that RAG-based methodologies dominate at scale, while MCP only excels when tool counts are within context limits.

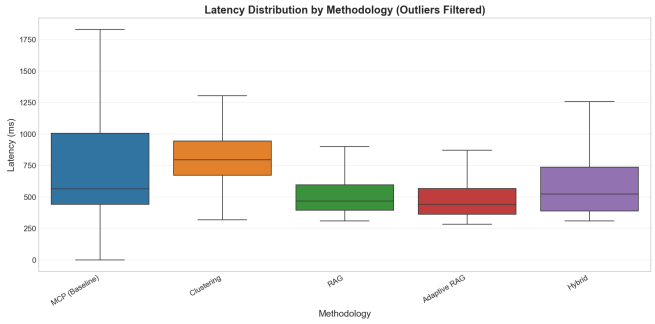

Figure 23: Latency Distribution by Methodology (Boxplot).

5.0.3 Latency Analysis. Figure 23 boxplot shows the distribution of end-to-end latency for each methodology, excluding outliers caused by network variability in cloud API calls.

Here is how the latency time is computed for each methodology:

- **MCP**: LLM API call time only
- **Clustering**: Sum of all LLM API calls across steps (category selection + tool selection + any backtrack steps)
- **Hybrid RAG**: RAG category retrieval time (embedding computation + similarity search) + LLM API call time
- **RAG**: RAG tool retrieval time (embedding computation + similarity search) + LLM API call time
- **Adaptive RAG**: Adaptive retrieval time (embedding computation + similarity search + adaptive k calculation) + LLM API call time

All methodologies exhibit similar median latency, which is dominated by LLM inference time. MCP shows higher variance due to the increase of context size when making LLM calls. Clustering has a higher mean due to multiple LLM calls across steps to first select

categories and then tools. RAG and Adaptive RAG demonstrate slightly lower median latency despite embedding computation overhead, because they make only one LLM call with fewer tokens in context. The embedding computation overhead of approximately 50-100ms is offset by faster LLM inference with smaller context.

5.0.4 Validation: Local LLM Experiments. To validate that our findings generalize beyond the cloud LLM setup and to enable complete dataset evaluation without API rate limits, we replicated key experiments using **Llama 3.2 3B running locally via Ollama**.

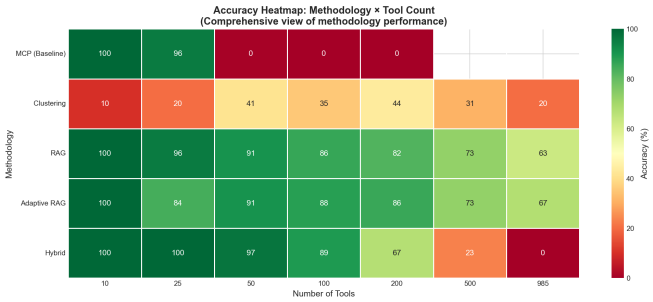

Figure 24: Local LLM (Llama 3.2 3B) Accuracy Comparison.

Despite the smaller model size and reduced context window, the relative performance rankings remain consistent: RAG-based methods outperform clustering, context limits affect MCP earlier (around 50 tools vs. 350 for the 70B model), and Adaptive RAG has a less clear advantage but remains competitive. This validation confirms that our methodology comparisons are not artifacts of the specific LLM used.

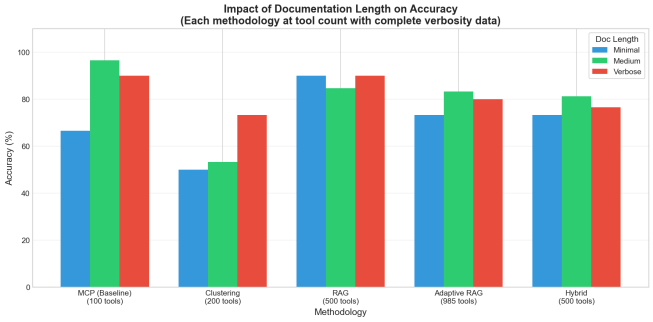

Figure 25: Verbosity Level Impact on Accuracy.

5.0.5 Verbosity Analysis. A small analysis on the effects of changing the verbosity level of tool descriptions on overall tool selection accuracy was conducted (Figure 25). Results indicate that:

- **MCP**: Significant accuracy drop (approx. 25-30%) when using short descriptions due to reduced context for the LLM.
- **Clustering**: A huge accuracy increase (approx. 20-25%) with has been noticed with verbose descriptions.
- **Hybrid**: Medium verbosity confirms to be the best choice, with approx. 5-10% accuracy drop on both sides.
- **RAG**: Minimal accuracy impact (approx. 5-7%) between verbosity levels, as retrieval focuses on key terms.
- **Adaptive RAG**: Slightly less robust than RAG, with approx. 10% accuracy drop using short descriptions and 4% with verbose ones.

Overall, verbosity has the largest impact on MCP and Clustering, while RAG-based methods are more resilient due to their reliance on retrieval rather than direct context. An interesting observation is that Adaptive RAG seems to benefit less from verbose descriptions compared to standard RAG, possibly because its adaptive k selection favors in some cases bigger contexts that confuse the LLM when verbosity is high. For this reason, except for RAG and Clustering, medium verbosity is generally the best choice across methodologies.

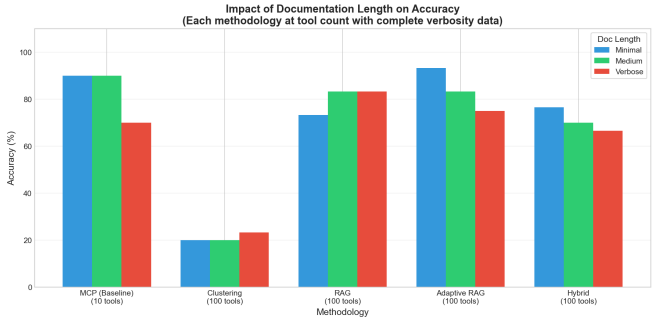

Figure 26: Verbosity Level Impact on Local Accuracy.

Interestingly, the local LLM experiments using Llama 3.2 3B via Ollama (Figure 26) show a different trend: only clustering and standard RAG show improvements with verbose descriptions. The other methodologies (MCP, Hybrid RAG, Adaptive RAG) all seem to drop in accuracy when verbosity is increased, possibly due to the smaller model struggling with larger contexts. This suggests that optimal verbosity settings may depend on the specific LLM being used, with larger models benefiting more from detailed descriptions.

Note: Verbosity analysis was only conducted with specific tool counts for each methodology, depending on the configuration (local or cloud). For cloud 985 tools were used for all methodologies except MCP (100 tools), while for local 100 tools were used for all methodologies except MCP (10 tools). This was done to accommodate context window limits and avoid results being skewed due to exceeding context.

5.0.6 Validation: xLAM Public Dataset. To ensure findings generalize beyond our custom tool definitions, we evaluated all methodologies on the **xLAM Function Calling 60K** dataset containing verifiable high-quality API descriptions and natural language queries. These experiment was tested with both Llama 3.3 70B via cloud API and Llama 3.2 3B running locally via Ollama.

To construct the tool definitions for xLAM, we parsed the dataset's API specifications and formatted them into the same YAML structure used in our synthetic dataset, including name, description, parameters, and test prompts. We then ran each methodology using the same experimental setup as before, adjusting context sizes and retrieval parameters as needed to accommodate the different

tool set. Additionally, parameters validation could be performed since the dataset includes detailed API parameter definitions and expected values. Of the 60,000+ APIs in the xLAM dataset, we selected a representative subset of **985** tools (for consistency) that required only single tool selection to answer their associated queries.

Note: Due to how the xLAM dataset is structured, it was not possible to test the Clustering-based two-step selection methodology, as the dataset does not include predefined categories for the tools.

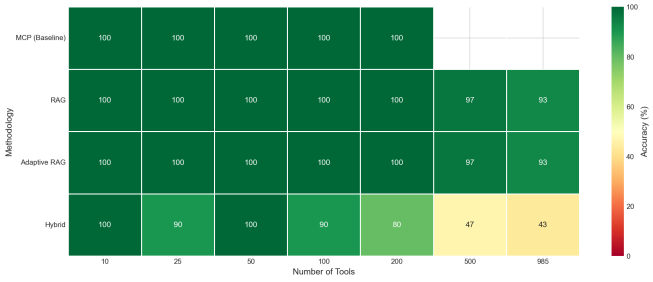

Figure 27: Cloud - xLAM Dataset Accuracy Comparison.

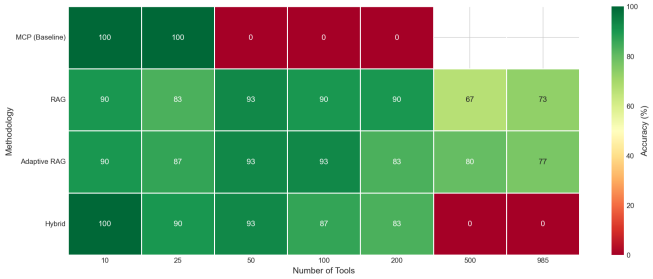

Figure 28: Local xLAM Dataset Accuracy Comparison.

Results on data shown in 27 and 28 align with previous dataset findings: RAG-based methods achieve the highest accuracy, just below MCP where context permits. The two experiments (cloud LLM and local LLM) show consistent trends, in addition to displaying how the model size and context window impact absolute accuracy.

These validation experiments provide confidence that our conclusions apply to practical deployment scenarios.

6 Considerations & Future Work

6.1 Limitations of This Study

6.1.1 Clustering Prompt Engineering Challenges. The clustering-based methodology's low accuracy stems partly from the difficulty of crafting effective category selection prompts. The LLM frequently misinterprets the category selection task, sometimes treating it as a general question-answering task rather than a tool selection task. While improved prompt engineering could potentially address this, we consider it a fundamental limitation of the hierarchical approach, the additional abstraction layer introduces failure modes not present in direct tool selection.

6.1.2 Experimental Aggregation Methodology. The charts and statistics presented aggregate data from 139 experimental configurations. While we strived for comprehensive coverage, not all methodologies were tested with identical parameter combinations. For instance:

- Verbosity analysis was conducted for specific tool counts for each configuration
- Prompt clarity comparisons focused on Adaptive RAG
- MCP could not be tested beyond 350 tools due to context limits

The trends reported are representative of observed performance, but precise numerical comparisons should account for configuration differences.

The shared source code also provides reports of more fair comparisons between methodologies on identical configurations, but do not include all tests conducted in this study (due to time constraints). These additional reports show similar trends to those presented here, confirming the generality of the findings.

6.1.3 Synthetic Dataset Simplicity. Our synthetic dataset uses relatively simple, targeted test prompts (e.g., "Generate a random number between 1 and 100"). While validation on the xLAM dataset provides some confidence in generalization, real-world queries are often more ambiguous, multi-part, or require interpretation. The accuracy figures reported may be optimistic compared to production scenarios with more complex user inputs.

6.1.4 Single Embedding Model. All RAG experiments used the *all-MiniLM-L6-v2* embedding model. While this provides consistency, different embedding models may yield different results. Domain-specific or larger embedding models might improve retrieval quality, particularly for specialized tool domains.

6.2 Future Work

6.2.1 Multi-Tool Selection. This study focused exclusively on single-tool selection scenarios. Many real-world agent tasks require invoking multiple tools in sequence or parallel. Extending the methodologies to handle multi-tool queries, where the system must identify all relevant tools and potentially their ordering, represents a natural next step.

6.2.2 Complex Prompt Evaluation. Future work should evaluate performance on:

- Ambiguous prompts that could match multiple tools
- Multi-step prompts requiring reasoning about tool combinations
- Adversarial prompts designed to confuse tool selection
- Conversational contexts where tool selection depends on dialogue history

6.2.3 Prompt Clarity Analysis. Preliminary results suggest that clearer, more explicit prompts improve accuracy. A systematic study of prompt engineering for tool selection, including few-shot examples, chain-of-thought prompting, and structured prompts, could yield practical guidelines for production deployments.

6.2.4 Embedding Model Comparison. Evaluating alternative embedding models (e.g., *all-mpnet-base-v2*, domain-specific models,

or instruction-tuned embeddings) could reveal accuracy improvements. The trade-off between embedding quality and computational cost merits investigation.

6.2.5 Adaptive K Heuristic Refinement. The adaptive k selection algorithm uses simple heuristics (elbow detection, threshold). More sophisticated approaches, potentially learned from data, could improve k selection. Reinforcement learning or meta-learning approaches that optimize k selection for specific tool domains represent promising directions.

7 Conclusion

This project systematically investigated strategies for scaling tool calling in LLM-based agents. Our experiments across 139 configurations and up to 985 tools reveal clear patterns in the efficiency-accuracy trade-off.

Key Findings.

(1) **MCP Hits a Hard Limit**: Naive context loading achieves high accuracy but becomes infeasible beyond 350 tools for our cloud configuration due to context window constraints. With 64K-token models, this approach cannot scale to enterprise-level tool ecosystems.
(2) **Hierarchical Selection Underperforms**: Clustering-based two-step selection dramatically reduces context usage but introduces a category selection bottleneck that limits overall accuracy to 40-50%. The cognitive overhead of hierarchical reasoning appears to exceed the context savings benefit.
(3) **RAG Enables Scalability**: RAG-based tool selection achieves the best balance, maintaining 80-90%+ accuracy while using only 2,000-5,000 tokens regardless of total tool count. This represents a 20-30x context reduction compared to MCP at scale.
(4) **Adaptive Selection Provides Marginal Gains**: Dynamically adjusting retrieval count based on similarity distributions yields slight accuracy improvements over fixed-k RAG, particularly at high tool counts, while maintaining or improving efficiency.
(5) **Verbosity Matters**: A correct tool description verbosity helps retrieval quality. Extremely brief descriptions hinder semantic matching, while overly verbose ones waste context budget without proportional accuracy gains, particularly for smaller models.

Closing Remarks. As LLM-based agents become integral to enterprise automation, the challenge of scaling tool ecosystems will only intensify. This study demonstrates that intelligent tool selection mechanisms, particularly RAG-based approaches, can bridge the gap between the expanding tool landscape and the finite context windows of current models. By treating tool selection as a retrieval problem rather than a context-stuffing problem, we can build agent systems that scale gracefully to thousands of tools while maintaining the accuracy users expect.

The code, datasets, and experimental configurations accompanying this report are available at https://github.com/sbrentan/tool-calling-optimization, enabling reproducibility and further exploration. The repository includes scripts to replicate all experiments, the datasets used, and detailed documentation on methodology implementation. Additionally, some reports of experiments conducted with different configuration plans are available, which can provide further insights into tool selection performance under varying conditions.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
[2] Yu Pan, Xiaocheng Li, and Hanzhao Wang. 2025. Online-Optimized RAG for Tool Use and Function Calling. *arXiv preprint arXiv:2509.20415* (2025).
[3] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems* 37 (2024), 126544–126565.
[4] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).
[5] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).
[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.